

15 Puzzle Backtracking Approach

Play the game: <https://15puzzle.netlify.app/>

Overview:

- **Backtracking** is a brute-force recursive algorithm.
- Explores all possible moves (states) from the current configuration.
- If a path leads to an invalid or previously visited state, it **backtracks** and tries a different path.

How it works:

1. Start from the initial state.
2. Recursively try all 4 possible moves (up, down, left, right).
3. Maintain a visited state list to avoid cycles.
4. If the goal state is reached, return the path.
5. Otherwise, backtrack.

Time and Space Complexity:

- **Time:** Exponential ($O(b^d)$, where b is the branching factor and d is the depth of solution).
- **Space:** Potentially huge due to the recursion stack and visited states.

Pros:

- Simple to implement.
- Guarantees a solution if one exists.

Cons:

- Extremely inefficient for large search spaces.
- No optimality: may not find the shortest solution.

DFS Approach with BFS Inspiration

Goal:

Use **DFS-style recursive backtracking** to explore all possible paths, but **track and return the shortest path**, similar to what BFS would find.

Key Concepts:

- **DFS recursion:** Explore all possible paths.
- **Visited set:** Prevent revisiting same states (to avoid cycles).
- **Shortest path tracking:** Store the shortest solution found so far.
- **Prune:** Stop recursion early if current path is already longer than the best solution.

Input:

- `initial_state`: 4x4 grid of the 15-puzzle (2D array).
- `goal_state`: target configuration.
- `moves`: valid directions (up/down/left/right).

Algorithm:

Function DFS_Shortest(state, path, visited, shortest_solution):

 If state == goal_state:

 If shortest_solution is empty OR length(path) < length(shortest_solution):

 shortest_solution ← copy of path

 Return

 If shortest_solution is not empty AND length(path) ≥ length(shortest_solution):

 Return // prune longer path

 visited.add(state)

 (x, y) ← coordinates of blank tile in state

 For each (dx, dy) in [UP, DOWN, LEFT, RIGHT]:

 nx ← x + dx

 ny ← y + dy

 If (nx, ny) is within grid bounds:

 new_state ← deep copy of state

 Swap blank with tile at (nx, ny)

 If new_state is not in visited:

 Append new_state to path

 Call DFS_Shortest(new_state, path, visited, shortest_solution)

 Remove last state from path (backtrack)

 visited.remove(state) // backtrack to allow different paths

Main Function:

1. shortest_solution ← empty list

2. visited ← empty set

3. path ← list containing only initial_state

4. Call DFS_Shortest(initial_state, path, visited, shortest_solution)

5. Return shortest_solution (if found)

☒ Python Code:

```
from copy import deepcopy

# Target goal state
GOAL = [
    [1, 2, 3, 4],
    [5, 6, 7, 8],
    [9, 10, 11, 12],
    [13, 14, 15, 0]
]

# Movement directions: Up, Down, Left, Right
MOVES = [(-1, 0), (1, 0), (0, -1), (0, 1)]

def is_goal(state):
    return state == GOAL

def find_blank(state):
    for i in range(4):
        for j in range(4):
            if state[i][j] == 0:
                return i, j

def state_to_tuple(state):
    return tuple(tuple(row) for row in state)

def print_state(state):
    for row in state:
        print(' '.join(f'{n:2}' for n in row))
    print()

def dfs_find_shortest(state, visited, path, shortest_solution):
    if is_goal(state):
        if shortest_solution[0] is None or len(path) < len(shortest_solution[0]):
            shortest_solution[0] = path.copy()
        return

    if shortest_solution[0] and len(path) >= len(shortest_solution[0]):
        return # prune longer paths

    visited.add(state_to_tuple(state))
    x, y = find_blank(state)

    for dx, dy in MOVES:
        nx, ny = x + dx, y + dy
        if 0 <= nx < 4 and 0 <= ny < 4:
            new_state = deepcopy(state)
            new_state[x][y], new_state[nx][ny] = new_state[nx][ny], new_state[x][y]
            state_key = state_to_tuple(new_state)
```

```

        if state_key not in visited:
            path.append(new_state)
            dfs_find_shortest(new_state, visited, path, shortest_solution)
            path.pop()

    visited.remove(state_to_tuple(state))

# Example scrambled state (solvable in a few moves)
initial_state = [
    [1, 2, 3, 4],
    [5, 6, 7, 8],
    [9, 10, 11, 12],
    [13, 15, 14, 0]
]

# Run the DFS + shortest tracking
visited_states = set()
shortest_solution = [None] # using a list to pass by reference
dfs_find_shortest(initial_state, visited_states, [initial_state], shortest_solution)

# Output the result
if shortest_solution[0]:
    print(f"✅ Shortest solution found in {len(shortest_solution[0]) - 1} moves:\n")
    for step in shortest_solution[0]:
        print_state(step)
else:
    print("❌ No solution found.")

```